

UnifAI / Raksha Enterprise Database Configuration & Persistence Guide

Audience: Database Administrators (DBA), DevOps Engineers, Platform Architects

Scope: PostgreSQL architecture, installation, user/database provisioning, **pg_hba.conf** security, SSL/TLS encryption, GORM schema management, table reference, PgBouncer pooling, **postgresql.conf** performance tuning, automated backup scripts, and disaster recovery.

Baseline: Repo commit **16d4c3e**

1. Database Architecture & Engine Support

UnifAI uses GORM as its persistence abstraction layer. The platform separates state into two logical stores:

- 1. **ConfigStore (framework/configstore):** Core configuration, identity, RBAC, access profiles, model providers, virtual keys, routing rules, prompts, skills, and settings.
- 2. **LogsStore (framework/logstore):** LLM inference logs, MCP tool execution traces, Browser AI employee prompt audit logs, search engine logs, and materialized aggregations.

Both stores can reside on the same PostgreSQL database or be split across dedicated database instances in high-scale environments.

Supported Database Engines

Engine	Production Recommended	Minimum Version	Notes
PostgreSQL	YES (Enterprise Standard)	14.x / 15.x / 16.x	Full JSONB support, GIN indexing, robust concurrency, streaming replication.
MySQL / MariaDB	Supported	8.0+ / 10.6+	Supported via framework/mysqlconn .
SQLite	NO (Development Only)	3.40+	Built via CGO go-sqlite3 (sqlite_static) . For single-developer local testing only.

2. PostgreSQL 16 Installation on Ubuntu / Debian

Execute as **root** or an administrative user with **sudo**:

```
# 1. Add official PostgreSQL APT repository sudo apt update && sudo apt install -y curl ca-certificates gnupg
lsb-release sudo install -d /etc/apt/keyrings curl -fsSL https://www.postgresql.org/media/keys/ACCC4CF8.asc
| sudo gpg --dearmor -o /etc/apt/keyrings/postgresql.gpg echo "deb
[signed-by=/etc/apt/keyrings/postgresql.gpg] http://apt.postgresql.org/pub/repos/apt $(lsb_release -cs)-pgdg
main" | sudo tee /etc/apt/sources.list.d/pgdg.list # 2. Install PostgreSQL 16 server, client, and contrib
extensions sudo apt update sudo apt install -y postgresql-16 postgresql-contrib-16 # 3. Enable and verify
service status sudo systemctl enable postgresql sudo systemctl status postgresql
```

3. Database & User Provisioning (Exact Project Schema)

Connect to the PostgreSQL console as the **postgres** superuser:

```
sudo -u postgres psql
```

Execute the exact provisioning SQL commands matching project **.env.example**:

```
-- 1. Create dedicated application user with SCRAM-SHA-256 encryption CREATE USER agent_unify WITH ENCRYPTED
PASSWORD 'change-me-strong-password'; -- 2. Create production database with UTF-8 encoding CREATE DATABASE
raksha_new WITH OWNER agent_unify ENCODING 'UTF8' LC_COLLATE 'en_US.UTF-8' LC_CTYPE 'en_US.UTF-8'; -- 3.
Connect to the newly created database \c raksha_new -- 4. Grant schema and default privileges GRANT ALL
PRIVILEGES ON DATABASE raksha_new TO agent_unify; GRANT ALL ON SCHEMA public TO agent_unify; ALTER DEFAULT
PRIVILEGES IN SCHEMA public GRANT ALL ON TABLES TO agent_unify; ALTER DEFAULT PRIVILEGES IN SCHEMA public
GRANT ALL ON SEQUENCES TO agent_unify; ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT ALL ON FUNCTIONS TO
agent_unify; -- 5. Install extensions required for UUIDs, full-text, and trigram search CREATE EXTENSION IF
NOT EXISTS "uuid-ossf"; CREATE EXTENSION IF NOT EXISTS "pg_trgm"; CREATE EXTENSION IF NOT EXISTS
"btree_gist"; -- Verify installed extensions \dx -- Exit psql \q
```

4. Network Security & **pg_hba.conf** Configuration

Edit **/etc/postgresql/16/main/pg_hba.conf** to enforce **scram-sha-256** password hashing and restrict incoming connections strictly to authorized hosts:

```
# TYPE DATABASE USER ADDRESS METHOD # Local administrative access via UNIX domain socket local all postgres
peer local all all scram-sha-256 # Localhost IPv4 loopback (when UnifAI runs on the same server) host
raksha_new agent_unify 127.0.0.1/32 scram-sha-256 # Localhost IPv6 loopback host raksha_new agent_unify
::1/128 scram-sha-256 # Docker Container Bridge Network (e.g. 172.16.0.0/12 or 10.0.0.0/8) host raksha_new
agent_unify 172.16.0.0/12 scram-sha-256 host raksha_new agent_unify 10.0.0.0/8 scram-sha-256 # Reject all
other unauthorized external traffic host all all 0.0.0.0/0 reject
```

Listen Address in **postgresql.conf**

Edit **/etc/postgresql/16/main/postgresql.conf**:

```
listen_addresses = 'localhost, 172.16.0.1' # Replace with your VPC private IP port = 5432 max_connections =
300
```

Reload PostgreSQL:

```
sudo systemctl reload postgresql
```

5. SSL/TLS Database Encryption (In-Transit Protection)

Enterprise production environments must encrypt all database traffic across the network.

5.1 Generating PostgreSQL SSL Certificates

```
sudo mkdir -p /etc/postgresql/ssl && cd /etc/postgresql/ssl # Generate 4096-bit private key sudo openssl
genrsa -out server.key 4096 sudo chmod 600 server.key # Generate self-signed certificate (or use corporate
CA) sudo openssl req -new -x509 -days 3650 -key server.key -out server.crt \ -subj
"/CN=db.internal.yourcompany.com" sudo chmod 644 server.crt sudo chown -R postgres:postgres
/etc/postgresql/ssl
```

5.2 Enabling SSL in postgresql.conf

```
ssl = on ssl_cert_file = '/etc/postgresql/ssl/server.crt' ssl_key_file = '/etc/postgresql/ssl/server.key'
ssl_min_protocol_version = 'TLSv1.2' ssl_ciphers = 'HIGH:!aNULL:!MD5'
```

Restart PostgreSQL:

```
sudo systemctl restart postgresql
```

5.3 Configuring UnifAI .env for SSL Mode

In your **.env** file, configure:

```
DB_TYPE=postgres DB_HOST=127.0.0.1 DB_PORT=5432 DB_NAME=raksha_new DB_USER=agent_unify
DB_PASSWORD=change-me-strong-password DB_SSL_MODE=require
```

Available SSL modes in GORM / pq driver:

- **disable**: Plaintext communication (development only).
- **require**: TLS encryption enforced; server certificate is not verified against CA.
- **verify-ca**: TLS encryption enforced; server certificate verified against trusted CA.
- **verify-full**: TLS encryption enforced; verifies certificate CA and validates server hostname.

6. Schema & Table Reference

UnifAI automatically runs GORM auto-migrations on startup (**framework/configstore/tables/migrations** and **framework/logstore/migrations.go**).

6.1 ConfigStore Tables (framework/configstore/tables/)

Table Name	Purpose	Key Columns / Schema Details
governance_users	User accounts & identity	id, username(uniq), email, password, role(admin/sub_admin/user), status(pending/approved/rejected/email_unverified/disabled), budget, rate_limit, allowed_prompt_repos, allowed_sections, reviewed_at, created_at
rbac_roles	RBAC role definitions	id, name(uniq), description, is_system_role, dac(all-data/own-data), permission_ids(JSON), created_at
access_pro_files	Policy templates	id, name(uniq), description, is_active, version, calendar_aligned, tags(JSON), spec(JSON), created_at
governance_business_units	Organizational units	id, name(uniq), team_ids(JSON), budget(JSON), rate_limit(JSON)
virtual_keys	Gateway API credentials	id, key_name, hashed_key, prefix, team_id, customer_id, budget, rate_limit, allowed_models(JSON), created_at

Table Name	Purpose	Key Columns / Schema Details
model_providers	AI provider credentials	id, name, provider_type, encrypted_api_key, base_url, weight, priority, is_active
models	Model catalog entries	id, model_name, provider_id, pricing_id, context_window, is_enabled
routing_rules	Dynamic model router	id, name, priority, condition_tree(JSON), target_model, is_active
circuit_breaker_policies	Upstream failure shields	id, name, failure_threshold, recovery_timeout_sec, half_open_requests
system_settings	Platform configuration	id, company_name, company_logo, allowed_origins, session_ttl_sec
smtp_configs	Email notification settings	id, host, port, username, encrypted_password, from_email, use_tls

6.2 Browser AI Tables (framework/logstore/browser_ai.go)

Table Name	Purpose	Key Columns / Schema Details
browser_ai_logs	Intercepted prompt audit trail	id, timestamp, platform, domain, user_prompt_full, action(ALLOW/BLOCK/REDACT/WARN), status, rule_triggered, risk_score, predictive_risk, agent_id, agent_hostname, client_ip, reply_bot_text, attachment_name, attachment_size, attachment_path, metadata(JSON)
browser_ai_search_logs	Search query telemetry	id, timestamp, engine, browser, is_incognito, query, clicked_url, clicked_title, agent_id, predictive_risk
browser_guard_rules	Guard DLP detection rules	id, name, pattern(regex), rule_type(regex/ai_bot), severity(CRITICAL/HIGH/MEDIUM), action(BLOCK/REDACT/WARN), warning_message, active, bot_provider, bot_model, bot_prompt
browser_ai_target_websites	Monitored AI platforms	id, domain(uniq), platform_name, monitored, block_entire_website, host_role(ui/chat/file), status(MONITORED/PAUSED/BLOCKED)
browser_ai_controls_settings	Global interaction policy	id, block_title, block_message, file_upload_warning, interaction_toggles(JSON)
browser_ai_agents	Fleet endpoint inventory	id, hostname, agent_type(endpoint/network), status(active/uninstall_pending/uninstalled), last_seen_at, health_detail, contact_email, uninstall_key_hash, uninstall_key_enc, proxy_bundle_sha
browser_guard_fleet_config	Agent heartbeat defaults	id, pac_sync_interval, fail_open, eval_timeout_sec, created_at
browser_guard_rebuilt_logs	Guard packaging logs	id, status(success/failed), bundle_sha, message, log, created_at

7. Connection Pooling with PgBouncer

In high-concurrency environments (over 500 active Browser Guard agents or 100k+ daily inference calls), use **PgBouncer** to pool database connections.

7.1 Installation

```
sudo apt install -y pgbouncer
```

7.2 Configuration (/etc/pgbouncer/pgbouncer.ini)

```
[databases] raksha_new = host=127.0.0.1 port=5432 dbname=raksha_new [pgbouncer] logfile =  
/var/log/postgresql/pgbouncer.log pidfile = /var/run/postgresql/pgbouncer.pid listen_addr = 127.0.0.1  
listen_port = 6432 auth_type = scram-sha-256 auth_file = /etc/pgbouncer/userlist.txt pool_mode = transaction  
max_client_conn = 1500 default_pool_size = 50 min_pool_size = 10 reserve_pool_size = 5 reserve_pool_timeout =  
5 server_idle_timeout = 300
```

Generate **/etc/pgbouncer/userlist.txt**:

```
"agent_unify" "SCRAM-SHA-256$..."
```

Enable and start PgBouncer:

```
sudo systemctl enable pgbouncer sudo systemctl restart pgbouncer
```

Point UnifAI **.env** to PgBouncer port:

```
DB_PORT=6432
```

8. Production PostgreSQL Performance Tuning (postgresql.conf)

Tune these parameters based on your database host RAM:

Recommended Settings for 16 GB RAM Server

```
# Memory Configuration shared_buffers = 4GB # 25% of total RAM effective_cache_size = 12GB # 75% of total RAM  
maintenance_work_mem = 1GB work_mem = 16MB # Per-query sort memory wal_buffers = 16MB # Checkpoints and Write  
Ahead Log (WAL) min_wal_size = 2GB max_wal_size = 16GB checkpoint_completion_target = 0.9 checkpoint_timeout  
= 15min # Disk I/O & SSD / NVMe Optimization random_page_cost = 1.1 # Fast SSD storage  
effective_io_concurrency = 200 # Concurrent asynchronous disk operations # Worker Processes  
max_worker_processes = 8 max_parallel_workers_per_gather = 4 max_parallel_workers = 8  
max_parallel_maintenance_workers = 4
```

9. Automated Daily Backup Script & Disaster Recovery

9.1 On-Demand Backup Command

```
# Custom compressed format backup (Recommended) pg_dump -h 127.0.0.1 -p 5432 -U agent_unify -Fc -b -v -f  
"raksha_new_$(date +%Y%m%d_%H%M%S).dump" raksha_new
```

9.2 Automated Daily Backup Script (/opt/unifai/scripts/db_backup.sh)

```
#!/bin/bash # ===== # UnifAI /
Raksha PostgreSQL Production Daily Backup Script #
===== set -e
BACKUP_DIR="/var/backups/unifai/db" DATE=$(date +%Y%m%d_%H%M%S)
BACKUP_FILE="${BACKUP_DIR}/raksha_new_${DATE}.dump" LOG_FILE="/var/log/unifai_db_backup.log"
RETENTION_DAYS=30 mkdir -p "${BACKUP_DIR}" echo "[$(date)] Starting UnifAI PostgreSQL backup..." >>
"${LOG_FILE}" export PGPASSWORD="change-me-strong-password" pg_dump -h 127.0.0.1 -p 5432 -U agent_unify -Fc
-b -v raksha_new > "${BACKUP_FILE}" 2>> "${LOG_FILE}" if [ $? -eq 0 ]; then SIZE=$(du -h "${BACKUP_FILE}" |
cut -f1) echo "[$(date)] Backup completed successfully: ${BACKUP_FILE} (${SIZE})" >> "${LOG_FILE}" else echo
"[$(date)] ERROR: Backup failed!" >> "${LOG_FILE}" exit 1 fi # Retention policy: remove backups older than 30
days find "${BACKUP_DIR}" -name "raksha_new_*.dump" -type f -mtime +${RETENTION_DAYS} -delete echo
"[$(date)] Cleaned up backups older than ${RETENTION_DAYS} days." >> "${LOG_FILE}"
```

Make executable and schedule in Crontab:

```
sudo chmod +x /opt/unifai/scripts/db_backup.sh # Add to root crontab (sudo crontab -e): 0 2 * * *
/opt/unifai/scripts/db_backup.sh >/dev/null 2>&1
```

9.3 Step-by-Step Disaster Recovery Restore Procedure

In the event of database corruption or hardware failover:

```
# 1. Terminate all active application connections to the database sudo -u postgres psql -c "SELECT
pg_terminate_backend(pid) FROM pg_stat_activity WHERE datname = 'raksha_new' AND pid <> pg_backend_pid();" #
2. Drop and recreate database sudo -u postgres psql -c "DROP DATABASE IF EXISTS raksha_new;" sudo -u postgres
psql -c "CREATE DATABASE raksha_new WITH OWNER agent_unify ENCODING 'UTF8';" # 3. Restore database schema,
tables, and data using pg_restore sudo -u postgres pg_restore -d raksha_new -v -0 -x
"/var/backups/unifai/db/raksha_new_20261003_120000.dump" # 4. Re-grant privileges to application user sudo
-u postgres psql -d raksha_new -c "GRANT ALL ON ALL TABLES IN SCHEMA public TO agent_unify; GRANT ALL ON ALL
SEQUENCES IN SCHEMA public TO agent_unify;" # 5. Verify restored tables and count sudo -u postgres psql -d
raksha_new -c "\dt"
```

End of Database Configuration Guide.